

Pioneer.jl SearchDIA Allocation Analysis

Allocation-reduction effort — branch `perf/alloc-diagnostics`
(off `perf/combined-allocations`)

2026-06-24

Abstract

This document tracks the memory-allocation analysis of Pioneer’s **SearchDIA** pipeline. Benchmark: `MtacYeastStandard3M` (3 Thermo Astral DIA files, 3-min gradient) searched against the `yeast_May7-2026_bitvec_10.poin` library (~619k target precursors), at 36 threads unless noted. All numbers are warm (post-JIT) unless stated. “Allocation” means cumulative heap bytes (`gc_bytes` / `@timed.bytes`, summed across threads), which is GC churn, *not* peak working set (RSS).

1 Methodology

Three complementary instruments were used:

- `@timed per-phase` (exact): true total bytes per pipeline phase.
- `@alloc_bucket` (exact, env-gated `PIONEER_DIAG_ALLOC`): `gc_bytes` delta around named Main Search sub-steps.
- `Profile.Allocs` (sampled, rate 10^{-4}): attributes each sampled allocation *event* to its leaf source line and to the phase (`SearchMethods/⟨X⟩`) its stack passes through.

Important caveat. `Profile.Allocs` samples allocation *events*, so it pinpoints frequent *small* allocations but *under-counts rare large* ones (a single multi-GB **resize!** is one event, usually missed). Hence the sampled per-line totals are much smaller than the exact `@timed` phase totals; the two are used together (exact = “how much per part”, sampled = “which line churns”).

2 Per-phase totals (exact, warm)

Total warm allocation on the combined branch: **35.0 GB** over 3 files.

Phase	Mem Alloc (GB)
Main Search	16.76
Precursor Scoring	4.06
Quantification & Output	3.52
Chromatogram Integration	2.54
BitVec Calibration	2.14
Protein Scoring	1.98
Parameter Tuning	1.86
Spectral Library Loading	0.82
Quadrupole Tuning	0.39
Search Context Initialization	0.37
Huber Calibration	0.34
Protein Inference	0.25

3 Main Search breakdown (exact @alloc_bucket, per file)

The post-deconvolution PSM table footprint is only 0.264 GB (200 B/row), rising to 0.363 GB (275 B/row) after features — so most per-file Main Search allocation is churn, not data footprint.

Sub-step	file 1	file 2	file 3
library_search (deconv)	6.61	0.55	0.52
train_lgbm_and_select_best	0.88	0.93	0.88
chromatogram_features	0.50	0.53	0.50
permute_by_precursor	0.30	0.31	0.30
ms1_lookup_features	0.23	0.24	0.23
prepare_psm_features	0.16	0.17	0.16
scan_competition_features	0.02	0.03	0.02

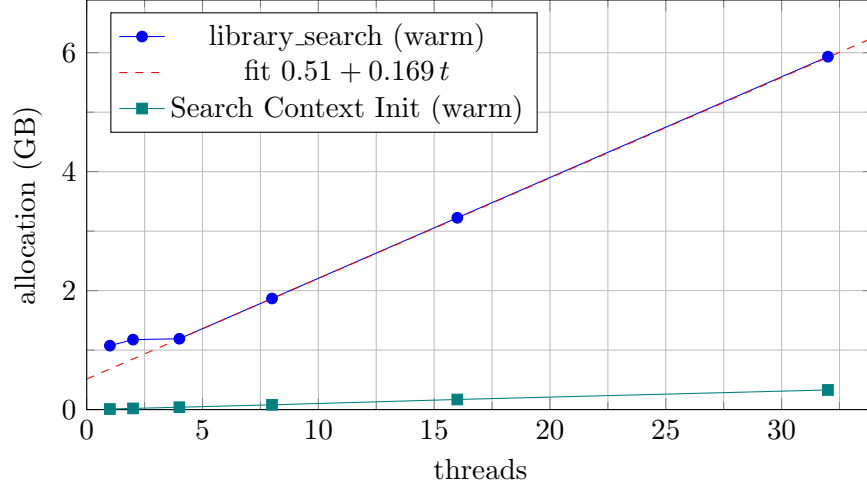
Key observation: `library_search` allocates 6.6 GB on file 1 but only ~ 0.52 GB on files 2–3. The excess is *one-time per-thread buffer growth* that is reused across files (Section 4). The *recurring* per-file Main Search cost is ~ 2.7 GB, led by `train_lgbm` (~ 0.9), `library_search` (~ 0.52), `chromatogram_features` (~ 0.5).

4 Thread-scaling of the one-time deconv growth

To test whether the file-1 growth is per-thread buffer allocation, the search was run on a *single* file at varying thread counts. “Ctx Init” is the **Search Context Initialization** phase (pure per-thread struct allocation); “library_search” is the file-1 deconv bucket. “r1” = cold (first **SearchDIA** in the process), “r2” = warm (second call; JIT already paid, fresh context so buffers regrow).

threads	CtxInit r1	CtxInit r2	libsearch r1	libsearch r2	r1–r2 (JIT)
1	0.05	0.01	1.648	1.075	0.573
2	0.06	0.02	1.748	1.176	0.572
4	0.08	0.04	1.763	1.191	0.572
8	0.12	0.08	2.440	1.868	0.572
16	0.20	0.17	3.797	3.225	0.572
32	0.37	0.33	6.505	5.933	0.572

All figures GB. `r1-r2` is constant (≈ 0.57 GB) across all thread counts: that is the thread-independent JIT compilation cost.



4.1 Decomposition and theory check

Search Context Init (per-thread struct allocation) is linear through the origin at 10.3 MB/thread (slope of the warm series). The theoretical per-thread footprint — `Base.summarysize` of one `SimpleLibrarySearch` minus the shared isotope splines — is 11.05 MB/thread. These match (the small gap is shared/padding that `summarysize` attributes but marginal `gc.bytes` does not). The init buffers are dominated by `mass_err_samples` (250,000 samples) and the `precursor_weights SparsePrecMap` (sizehint 262,144).

`library_search (deconv)` fits (threads ≥ 4):

$$\text{alloc}(t) \approx \underbrace{0.51 \text{ GB}}_{\text{thread-independent}} + \underbrace{0.169 \text{ GB}}_{\text{per thread}} \cdot t. \quad (1)$$

Interpretation:

- The **constant** 0.51 GB is the per-file, per-scan deconv work (DataFrame build, per-scan temporaries). It is data-bound, thread-independent, and *recurs every file* — exactly matching the ~ 0.52 GB observed on files 2–3.
- The 0.169 GB/thread is per-thread scratch *growth* during deconvolution: the fused design matrix (`SparseArrayFused/Hs`), the scored/unscored PSM buffers and `FusedScratch resize!` grow from their seed sizes (5000 / 256 elements) to the largest scan’s complexity. It is paid *once per SearchDIA call* (on file 1) and reused by later files.

At 36 threads this predicts $0.51 + 0.169 \cdot 36 = 6.6$ GB, matching the observed file-1 `library_search`.

Per-thread totals: ~ 11 MB at init (matches theory) growing to ~ 180 MB of cumulative deconv allocation per thread. The growth is the working set (~ 85 MB/thread after geometric `resize!`, $\sim 2\times$ in cumulative bytes) sized to the busiest scan.

4.2 Consequences

- The one-time growth is **strongly thread-dependent** ($\propto$ threads). It amortizes to zero over many files (constant working set as files grow — the intended scalability property), so it is *not* a priority for many-file runs, but it dominates short/single-file runs.

- To shrink it: seed the deconv scratch closer to the typical max-scan size (fewer `resize!` doublings $\Rightarrow$ less cumulative churn), and/or reduce the per-thread buffer count. Lower thread counts reduce cumulative allocation linearly at the cost of parallelism.

5 Per-phase top allocation sites (sampled)

Frequent-small-allocation churn sites within each phase (`Profile.Allocs`).

Main Search

`features.jl` feature loops (macro-expansion lines 1043–1046, 1469–1475, ~ 0.5 GB total); `_ms1_lookup_scan_runs` (`features.jl`:728/737); `add_psm_features!` (`features.jl`:93); `_ms1_ms2_explained_delta` (`features.jl`:425); `count_token_ids!` (`irt_refinement.jl`:215, Arrow-string iteration remainder).

Quantification & Output

`writeCSVTables.jl`:511 CSV row writing (0.42 GB / 22M allocs); `build_accession_to_species` `split()` (`maxLFQ.jl`:653–660, ~ 0.43 GB); `fillColumn!` (`MergeOperations.jl`:67, 0.30 GB — type-unstable Arrow column index).

Precursor Scoring

`wide_window_features.jl` (lines 385/99/558/840/844, ~ 0.45 GB); `pass1_oom.jl` sampling; `PipelineOperations` column adds.

Protein Scoring

`_build_protein_rollup` (`protein_inference_pipeline.jl`:438–540, ~ 0.6 GB spread); `count_protein_peptides` (`run_protein_scoring.jl`:39/48, 0.29 GB).

Chromatogram Integration

`build_chromatograms` (`utils.jl`:854, 0.23); `WHSmooth!` (`integrate_chrom.jl`:269/273/279); `process_final_psms!` (`utils.jl`:1203).

Parameter Tuning

`save_multipage_pdf` (`pdfUtils.jl`:133, 0.32, QC plot rendering — intrinsic); `probit fill $\eta$ !` (`probitRegression.jl`:87, 0.27).

6 Optimization opportunities (ranked)

1. `library_search` per-thread scratch growth (0.169 GB/thread): pre-size the fused design matrix / PSM buffers (one-time; amortizes over files).
2. `train_lgbm_and_select_best` (~ 0.9 GB/file, recurring): audit the Julia-side feature-matrix build for reusable buffers.
3. `_build_protein_rollup` (~ 0.6 GB), `count_protein_peptides` (0.29 GB): per-protein temporaries $\rightarrow$ scratch reuse.

4. `build_accession_to_species` (0.43 GB) / `writeCSVTables` (0.42 GB): redundant per-precursor `split()`; CSV row churn.
5. `features.jl` feature-loop churn (~ 0.5 GB).

7 Changes already committed (branch feature/reduce-allocations)

Commit	Change
96b0d451e-da80c8dda	iRT refinement redesign (hardcoded token-id map + scratch + parser)
4a03a4713	type-stabilize <code>EmitToAccumulator/PatternAccumulator</code> (BitVec)
6070469c7	function barrier in <code>pass1_oom</code> sampled-column copy
2043cadbb	inline per-scan closure in <code>process_scans_fused</code>
cfee665c3	per-thread scratch reuse in fragment-correlation features

Cumulative (cold, vs develop baseline): Main Search $33.4 \rightarrow 24.5$ GB (-27%); TOTAL $82.9 \rightarrow 72.4$ GB. The combined branch (with parallel work: `getMz/getIntensity` type-stability, MS1 chromatogram scratch, scored-PSM buffer growth, single-NCE vcat skip) is at 35.0 GB warm. Result-neutral except one approved rebaseline of the iRT-refinement column order ($154,013 \rightarrow 154,874$ precursors).